

GROUP PROJECT (30%)

TechFix – Device Repair & Service Centre Management

A. Introduction

This is a GROUP project that contributes 30% to the total grade of this course. Each group shall consist of 4 or 5 members

B. Objectives

- To evaluate students' ability to design classes and implement an object-oriented program for a given problem.
- To construct a C++ program using different types of data structures of their choice to store and manage data.
- To utilize searching and sorting algorithms to support data manipulation.
- To analyse the impacts on the performance of the program based on the selected data structure and algorithm design.
- To justify the chosen solution clearly and confidently.

C. Project Description

TechFix is a busy electronics repair shop operating with a fixed inventory of spare parts. When devices arrive, staff process incoming requests by recording customer details, device models, and required service tasks. Based on these details, the system should allow staff to sequence the repair jobs (e.g., managing a first-come, first-served waitlist or prioritizing urgent fixes), track the overall service progress as individual tasks are completed and call customer for picking up the completed repair. You are tasked with designing how the system processes this repair flow, handles the sequencing of waitlisted jobs, and organizes the inventory and customer data.

D. Project Requirements

To improve operations, your group is tasked with designing an object-oriented C++ program to manage these tasks systematically. Your program should demonstrate your understanding and application of Object-Oriented design principles (Inheritance, encapsulation, and polymorphism) and custom-built data structures and algorithms.

IMPORTANT RESTRICTION: You are **NOT ALLOWED** to use predefined data structures provided in the C++ library (such as STL vector, stack, queue, list, etc.). All data structures must be implemented from scratch.

i. Your system should provide the functions listed below continuously based on user selection:

- Menu: Display main menu options.
- Data Management (Add, Update, Delete, View): Your system must handle various data entities (Repair job tickets, repair progress, customer details, fixed repair services, etc). You are free to design how to store this data, provided your overall system implements multiple lists utilizing ALL of the following:
 - At least ONE Array-based list.
 - At least ONE Linked List.
 - At least ONE Stack OR Queue (e.g., process incoming broken devices, list of repair tasks, Urgent Repair, etc). The system must use a strict Queue (FIFO) or Stack (LIFO) concept in

simulating the data process, with zero direct-index access. (Note: You must thoughtfully design and decide which specific data structure is best suited for each specific feature, and you will justify these choices in your report).

- **Searching Techniques:** You must implement BOTH Linear Search and Binary Search within your system. You are completely free to decide where to apply them ((e.g., searching for a specific Customer Job ID to update their repair progress, searching the inventory list to check for available repair parts, etc.)). However, both algorithms must be present, functional, and justified.
- **Display Data Lists (View Operations):** The system must provide basic CRUD functionality to display the full list of recorded entries for at least two distinct data entities (e.g., repair job tickets, customer details, or spare parts inventory, etc). These list displays must vary from the Summary Report.
- **View Summary Report (Sorting Techniques):**
 - Generate an Active Job Routing & Task Report as shown in Figure-1.
 - Implement a suitable sorting algorithm (e.g., Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, or Merge Sort).
 - You must provide flexibility to choose sorting keys, allowing the report to be sorted by Job ID (Default Sort) or by Repair Progress.

Job ID	Customer Device	Current Job Status	Repair Progress	Repair Tasks
J-001	iPhone 13	Closed	Completed	Hardware Diagnostics, Water Damage Cleaning
J-005	iPad Pro 11"	Ready for Customer Pickup	Completed	Hardware Diagnostics, Battery Replacement, Quality Check
J-003	MacBook Air M2	Pending Initial Diagnostics	In Progress	Hardware Diagnostics, Fan Replacement
J-002	Samsung Galaxy S22	Queued for Specialist: Screen Repair Technician	In Progress	Hardware Diagnostics, Screen Replacement, Quality Check
J-004	Dell XPS 15	Queued for Specialist: Motherboard Technician	Pending	Motherboard Micro-soldering, OS Restoration

Figure-1: TechFix Active Job Routing & Task Report (Sorted by Repair Progress)

ii. General System Constraints & Best Practices:

- **Design Justification:** You must thoughtfully plan your solution and be prepared to justify why a specific data structure or algorithm was chosen for a specific task over alternatives.
- **Pre-set Data:** Create at least five (5) pre-set (hard-coded) records for repair tickets and parts so that the complete data is accessible and manipulatable immediately upon running the program.
- **Input Validation & User-Friendly Design:** Implement comprehensive input validation throughout the program run, ensuring the system checks for logic errors and out-of-range data. The program must produce relevant outputs with excellent design and clear user instructions.
- **Good Programming Practices:** Produce a readable program by ensuring comprehensive comments in the code, consistent use of white spaces, proper indentation, and consistent naming conventions.

E. Project Presentation

All group members MUST take part in the presentation and present professionally. Importantly, your presentation will be heavily evaluated on Understanding of Program Design and code implementation.

F. Project Peer Evaluation

Every group member must evaluate their teammates by completing the Peer Review Form (Refer to Appendix A) and submitting it AFTER the project presentation. The final project mark for an individual student is determined by their percentage in the peer evaluation. (The submission channel will be announced by your lecturer later.)

G. Submission & Deliverables Requirements

i. Submission Deadline and Platform

- All project deliverables must be submitted via the link provided on **eLEAP by 24 May 2026 (Sunday) at 11.59 pm**. 20% marks deduction per day for late submission.

ii. Required Deliverables

- C++ Source Codes (*.h, *.cpp): All working source code files. Make sure your codes are fully compliable and executable in Embarcadero DevC++ IDE. Code that cannot compile will lose marks.
- Presentation Slides (*.pptx): The slides your group will use for the live presentation.
- Project Report (*.pdf): A consolidated report document that MUST include the following elements:
 - *UML Class Diagram*: Illustrating your Object-Oriented design.
 - *System design, implementation, and justification*: Explain the system design, detail which specific data structures and algorithms handle which data and justify the rationale behind your free design choices and their impact on performance.
 - *Snapshots of functional outputs and error validation*: Include complete screen snapshots of all possible functional outputs and implemented error validations, along with a brief, clear explanation for each.
 - *Special information*: Any special instructions (passwords, operation steps that must be followed, etc.).
 - *Evidence of Collaboration*: Clear at least 2 snapshots/screenshots showing evidence of your team's ONLINE collaboration (e.g., social media chats, emails, or collaboration tool screenshots like Webex/MS Teams). *Note: Photos of physical or face-to-face discussions will NOT be accepted.*

iii. Report Format:

- Page limits: Should NOT exceed ten (10) pages (excluding the front cover).
- Font Type - Times New Roman (Font Size – 12pt) or Calibri (Font Size – 11pt); Line Spacing - 1.5; and paragraph text arrangement - justified.

- The font size displayed in the screenshots and UML diagram should be similar to report font size.
- All images and diagrams should be clear with readable fonts and well edited (show only the relevant information).

iv. File Formatting and Naming Convention

- Submit your deliverables in a single zipped file format (.zip). Files submitted in incorrect formats (such as .rar or .7z) will not be entertained.
- Please use the following naming format for your submission file: [LecGroup_ProjGroupNo#.zip](#) (Example: *LG1_ProjGroup12.zip*)
- Each group is required to submit only **ONE (1)** complete set of the project deliverables. Please designate one group member to make the final upload.

v. Academic Integrity & Plagiarism

- Any form of plagiarism (code similarity above 50% between two files or groups) will result in an automatic **zero (0) mark** for both parties involved.

APPENDIX - TMF1434 PROJECT PEER EVALUATION FORM

Important Grading Notice:

- a. Student final mark for the project is strictly determined by the percentage received in this peer evaluation
- b. Each group member **MUST individually** fill out this form to evaluate their teammates. This form is strictly confidential.
- c. Please submit your peer evaluation form only after presenting your project, as unforeseen circumstances may arise that could affect your evaluation at the last minute, e.g., a member may be unexpectedly absent or perform exceptionally well during the Q&A etc. (The submission channel will be announced by your lecturer later.)

Student Name (Evaluator):			
Matric No.:		Team Size:	<i>students</i>
Lecture Group No.:		Project Group No.:	

Rate each member on a scale of **0 (Terrible)** to **5 (Excellent)** for each criterion based on the rubric below. **DO NOT** evaluate yourself (You may refer to the **Point Scale Guideline** in the next page to determine appropriate scale in your evaluation). Make sure the **Name** and **Matric number** are correctly filled.

Teammates Being Evaluated		Member 1	Member 2	Member 3	Member 4	Member 5
Name						
Matric No						
What to assess	Question to ask:					
Task Execution & Responsibility (Willingness to take on & complete tasks)	Did this member willingly take responsibility for their assigned tasks and complete them without forcing other team members to chase them or do the work for them?					
Quality of Work & Timeliness (Accuracy, usability & meeting deadlines)	Was the work they submitted of high quality, and did they deliver it strictly on time so the rest of the team could proceed without bottlenecks?					
Commitment & Responsiveness (Attendance, participation & reliability)	Did this member consistently attend meetings, respond to group messages promptly, and never go 'Missing In Action' (MIA) when the team needed them?					
Collaboration & Communication (Teamwork, helpfulness & clarity)	Did this member work well with the team, willingly help others when issues arose, and communicate clearly to avoid any confusion?					
System Mastery & Willingness to Learn (Understanding design, algorithms & code)	Did this member show a strong effort to learn the system design, architecture, and code implementation (even the parts they didn't write) so they were fully prepared for the final presentation?					
Total Marks		/ 25	/ 25	/ 25	/ 25	/ 25

2. Point Scale Guideline (Evaluation Rubric)

Use the following rubric to determine the appropriate point value (0-5) for each criterion.

Scale Criteria	0 - Terrible (Zero Contribution)	1 - Very Poor (Highly Unreliable)	2 - Poor (Needs Improvement)	3 - Fair (Bare Minimum)	4 - Good (Solid & Reliable)	5 -Excellent (Exceptional Performance)
Task Execution & Responsibility (Willingness to take on and complete tasks)	Contributed absolutely no work. Acted as a complete "free-rider" who forced others to do their work.	Barely contributed. Ignored their assigned tasks and avoided taking responsibility.	Failed to execute assigned tasks properly or needed to be constantly chased by the team.	Completed assigned tasks but did the bare minimum. Sometimes required reminders.	Took responsibility for their share of tasks and completed them without needing to be chased.	Highly responsible. Completely executed their assigned project tasks willingly and perfectly without any reminders.
Quality of Work & Timeliness (Accuracy, usability, and meeting deadlines)	Submitted no work or submitted completely broken/AI-generated junk.	Submitted extremely poor-quality work and constantly missed deadlines, causing major delays.	Submitted work with major errors that others had to fix. Frequently missed deadlines.	Work had some minor errors that needed fixing, or the delivery was occasionally delayed.	Completed tasks on time with good quality and only minor errors.	Flawless delivery. All assigned tasks were completed with excellent quality and delivered perfectly on time.
Commitment & Responsiveness (Attendance, participation, and reliability)	Skipped all meetings, ignored all messages, and was completely "Missing In Action" (MIA).	Almost always MIA. Highly unreliable in group communications and project planning.	Frequently went MIA, skipped meetings, or ignored group discussions.	Attended most meetings and participated but was occasionally slow to respond to messages.	Generally committed, attended scheduled meetings, and was a reliable communicator.	Highly committed. Never went MIA, always participated actively in meetings, and was consistently responsive.
Collaboration & Communication (Teamwork, helpfulness, and clarity)	Refused to collaborate, did not help the team at all, and caused major project confusion.	Ignored team communications entirely and was highly unhelpful during the project's integration.	Caused confusion due to poor communication; mostly kept to themselves rather than helping others.	Collaborated reasonably well but sometimes lacked clarity, which caused minor team confusion.	Communicated well, collaborated effectively, and supported the team's efforts without issues.	Flawless teamwork. Worked perfectly with the team, always willing to help others, and communicated with total clarity.
System Mastery & Willingness to Learn (Understanding design, algorithms, and code)	Has zero understanding of the project. Showed absolutely zero effort or willingness to learn.	Shows almost no effort to understand the project architecture, putting the live defense at risk.	Weak understanding of the system design and algorithms. Showed little effort to learn the project.	Basic understanding of the system. Showed some effort but might struggle to explain deeper architecture.	Good understanding of the overall project design. Put in effort to learn parts of the system they didn't write.	Deep mastery. Fully knows the system design, algorithms, and code. Showed immense effort to learn the entire project.